RSIFlow: Adaptive Self-Improvement Workflows with an Evolving Meta-Agent

随瓶颈选择改进路径，并让改进者积累经验

Huan Zhang, Mingju Chen et al. · arXiv:2610.21791v1 · 2026

背景

核心问题：任务智能体变了，改进它的方法也应随之变化。 RSIFlow 根据当前瓶颈选择改模型、改框架或修成果，并把干预经验沉淀到元智能体中。

自我改进不只有模型训练

代码生成、工具使用和多跳问答的错误，可能来自模型能力、执行框架，也可能只是当前产物有缺陷。不同原因需要不同干预。

组件级方法固定优化一个部件；系统级方法协调多个部件；递归自我改进还要改进“如何产生下一次改进”的机制。

两个相互关联的瓶颈

一次有效更新会改变后续短板。始终采用同一工作流，可能继续修改已经不再是主要瓶颈的组件。

另一方面，编排改进流程本身也需要学习。成功与失败的干预都能提供诊断、选择及执行经验，不能执行完就丢弃。

图 1 • 从固定更新转向递归反馈

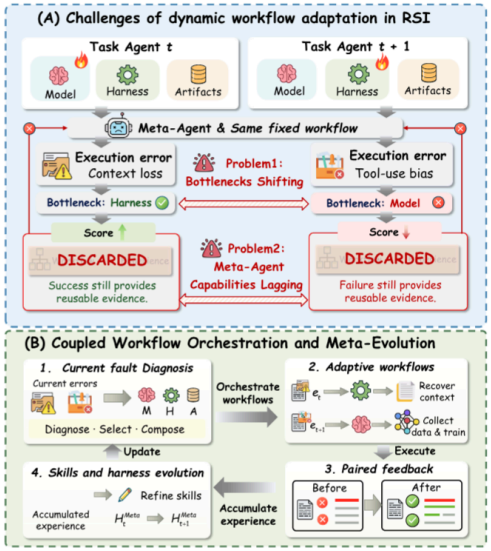


Figure 1: **Motivation of RSIFlow.** (A) Two coupled challenges in RSI. (B) The execution loop of RSIFlow.

图 1：瓶颈会移动，工作流经验应反馈到元智能体。

循环依次执行故障诊断、选择工作流、比较父代与候选，再更新技能和元框架。下一轮既使用更新后的任务智能体，也使用改进后的编排经验。

相关工作

本文把“选择改哪个组件”和“改进选择与编排能力”结合起来，区别于固定组件更新或预设交替顺序。

组件优化与固定联合更新

STaR 改模型参数，AFlow、DSPy、GEPA 等优化工作流或提示，AlphaEvolve 改执行产物；Reflexion、Voyager 则积累反思或可执行技能。

Co-Harness、HarnessForge 交替修改模型与框架，着重改善兼容性；其预设更新顺序未直接解决随瓶颈改变干预目标的问题。

适应性选择与元层递归

SIA 用轨迹反馈选择框架或权重更新，但选择器依赖冻结模型先验；HASE 把解题与框架编辑纳入同一动作空间。

Hyperagents 与 MetaSkill-Evolve 让改进者或改进技能本身可演化。RSIFlow 则用工作流结果持续修订元智能体的技能和执行接口。

方法

方法

任务层选一种组件更新，元层总结所有尝试。 元模型参数保持冻结，递归演化发生在元智能体的 harness 和技能库中。

图 2a • 任务智能体与元智能体

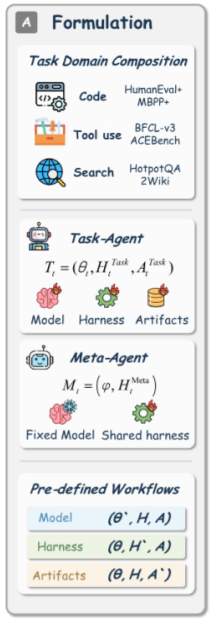


图 2A：任务领域、Task-Agent、Meta-Agent 与三种候选更新。

任务智能体包含模型参数、执行框架和任务产物；元智能体负责诊断与改进编排。两者都用 Action、Skill、Planning 描述框架。

“元智能体演化”并非训练元模型权重：其参数 ϕ 固定，改变的是执行接口、技能调用和指导流程。

$$T_t = (\theta_t, H_t^{\text{Task}}, A_t^{\text{Task}}),$$
$$M_t = (\phi, H_t^{\text{Meta}}), \quad X_t = (T_t, M_t)$$

θ 是任务模型参数；H 是框架；A 是代码或解决方案等任务产物。

图 2b–c • 按瓶颈选择一次更新

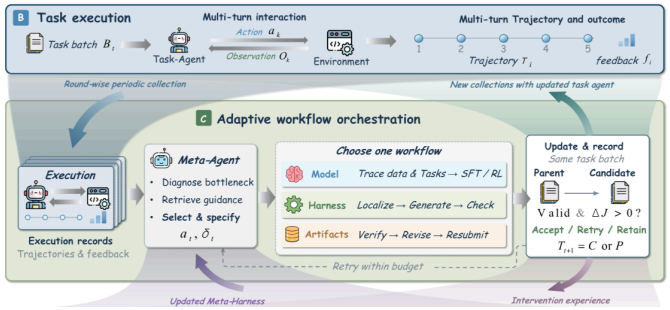


图 2B–C：执行任务、诊断瓶颈、选择工作流，并以同批次比较父代与候选。

元智能体读取执行轨迹、环境反馈和已有技能，在 Model、Harness、Artifacts 中选择一个目标；其余两个组件保持不变。

候选与父代使用同一任务批次比较。预算内遇到第一个有效且增益为正的候选就接受，否则保留父代；全部尝试都进入后续经验提炼。

路径一：更新模型参数

默认从成功轨迹构造数据做 SFT，复用已有执行经验，降低新增探索成本。候选继承父代的框架和产物，只更换模型参数。

$$\min_{\theta} \mathcal{L}(\theta; \mathcal{D}_t) = \mathbb{E}_{z \sim \mathcal{D}_t}[\ell(\theta; z)]$$

z 是样本或样本组，ℓ 为所选训练方法的损失；正文默认 SFT。

路径二：修改执行框架

在 Action、Skill、Planning 中选定需修改的组件，依次定位、修订、生成与验证。可改变提示、上下文、记忆检索、规划规则、工具编排或内部检查。

修改必须遵守接口和权限约束；模型参数冻结。一次 Harness 更新可以联合调整多个框架子组件。

路径三：修复当前任务产物

元智能体给出诊断指导，由任务智能体修订代码或解答，并执行、检查和重新提交。元智能体不直接代改产物，模型与框架保持固定。

正增益接受规则只用于训练批次，不在评测集上按收益挑选候选。有效性检查与预算限制共同约束尝试。

$$\Delta J_t^{(j)} = J(\hat{T}_t^{(j)}; B_t) - J(T_t; B_t)$$

在同一训练批次 B_t 上比较；有效且 ΔJ>0 才接受，否则重试或保留父代。

方法 • 实验

图 2d • 把干预结果变成可复用经验

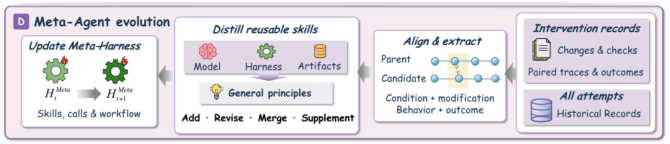


图 2D：对齐干预结果、提炼技能和原则、更新 Meta-Harness。

按共享子目标、工具事件或错误反馈对齐父代与候选轨迹，把修改、行为变化与最终结果联系起来；既提炼组件专属技能，也提炼跨组件原则。

每条记录包含适用条件、修改范围、条件化指导、应保持的约束、预期行为，以及支持和反对证据。

$$k = (\Omega_k, a_k, \rho_k, \Gamma_k, \hat{r}_k, E_k)$$

依次表示适用条件、范围、指导、保持约束、预期响应与证据。

维护技能库，再接入执行流程

Add 增加新策略；**Supplement** 追加证据；**Revise** 修改条件或指导并保留版本联系；**Merge** 合并重叠记录并保存来源。

更新后的知识要接入 Action 接口、检索调用与执行指令，才能改变下一轮诊断和候选生成。技能库不是只保存一段总结文字。

$$\mathcal{K}_{t+1} = (\mathcal{K}_t \setminus R_t) \cup C_t^+$$

R_t 为被替换或合并的旧记录，C_t^+ 为新增及更新记录；保留证据与修订历史。

实验

六个基准、两种任务模型规模检验跨领域收益；消融和同一初始任务智能体上的元版本比较，用于分离两层改进的贡献。

领域、模型与演化预算

代码：HumanEval+ 、MBPP+；工具：BFCL-v3 、ACEBench；检索问答：HotpotQA、2Wiki。基线覆盖模型单独更新、框架单独更新及联合 / 元层方法。

Task-Agent 为 Qwen3-4B 或 8B；Meta-Agent 为 GPT-5.6 Sol。主实验演化 **5 轮，每轮至多 2 次尝试**。

表 1 • 12 个设置中的 11 个领先

Table 1: Main results across six benchmarks with Qwen3-4B and Qwen3-8B backbones. Best results are highlighted in bold.

Method	Code		Tool Use		SearchQA		Overall
	HumanEval+	MBPP+	BFCL-v3	ACEBench	Hotpot	2Wiki	
Qwen3-4B							
AFlow	39.63	53.37	26.32	34.00	52.59	38.45	40.73
MermaidFlow	48.78	57.87	28.19	31.58	56.32	38.79	43.59
SFT	39.63	55.62	26.88	29.82	53.52	41.24	41.12
GRPO	43.90	55.06	22.30	37.25	54.39	39.32	42.04
HarnessForge	54.27	62.36	22.39	25.03	52.38	30.91	41.22
SIA	53.66	60.11	29.00	33.91	52.51	41.68	45.15
Hyperagent	43.90	52.81	26.88	31.57	51.82	40.59	41.26
RSIFLow	**68.29**	**66.29**	**29.23**	**38.00**	**56.72**	**48.72**	**51.21**
Qwen3-8B							
AFlow	68.29	69.10	34.89	51.46	53.55	48.32	54.27
MermaidFlow	69.51	68.54	30.96	46.20	56.29	42.70	52.37
SFT	68.29	69.10	37.01	40.35	51.57	44.49	51.80
GRPO	70.12	69.66	34.31	38.59	49.81	46.33	51.47
HarnessForge	70.12	67.42	38.15	**59.07**	45.83	37.12	52.95
SIA	73.78	70.22	40.13	51.46	53.96	46.43	56.00
Hyperagent	71.34	69.10	34.31	44.44	53.47	48.73	53.57
RSIFLow	**76.34**	**74.10**	**41.36**	52.62	**62.67**	**52.08**	**59.86**

表 1：全部 4B、8B 结果；Code 为 pass@1，SearchQA 为 F1，各基准独立比较。

整体分数为 **51.21%（4B）/ 59.86%（8B）**，分别比最强整体基线高 6.06 / 3.86 个百分点。4B 的 HumanEval+ 从最强基线 54.27% 提高到 68.29%。

例外是 **8B 的 ACEBench**：RSIFlow 为 52.62%，低于 HarnessForge 的 59.07%。不能把整体表现或 11/12 的领先概括为每一项都最好。

实验

表 2 • 自适应选择和元演化缺一不可

Table 2: Module ablation of RSIFlow using Qwen3- 4B under $R = 5$ evolution rounds. With meta evolution enabled, we compare fixed task-evolution routines targeting the harness (H), model (M), or artifacts (A).

Meta *Evo.*	Task *Rou.*	Code	Tool use	SearchQA
Vanilla		48.25	29.70	34.52
×	✓	56.73 (↑8.48)	29.70 (→0.00)	49.98 (↑15.46)
✓	×(H)	63.42 (↑15.17)	32.93 (↑3.23)	44.89 (↑10.37)
✓	×(M)	50.31 (↑2.06)	32.15 (↑2.45)	44.73 (↑10.21)
✓	×(A)	52.34 (↑4.09)	30.69 (↑0.99)	52.57 (↑18.05)
✓	✓	**67.29** (↑19.04)	**33.62** (↑3.92)	**52.72** (↑18.20)

表 2：Qwen3-4B、5 轮；括号为相对 Vanilla 的变化，H/M/A 表示固定更新组件。

完整方法的 Code / Tool Use / SearchQA 为 **67.29 / 33.62 / 52.72**。禁用元演化后为 56.73 / 29.70 / 49.98，三领域平均下降约 5.74 个百分点。

固定只改 H、M 或 A 各有偏科；自适应编排的三个领域得分均更高。原表保留了相对 Vanilla 的变化，不应与跨论文主表混比。

图 3 • rollout 预算与性能

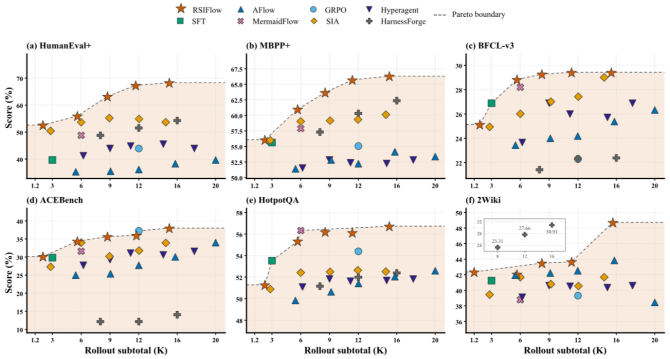


Figure 3: Budget–performance Pareto analysis across all six benchmarks

图 3：六个任务的 rollout—性能曲线，单位 K；星号为 RSIFlow。

六个任务中，RSIFlow 大多处于或接近经验 Pareto 前沿。轨迹同时服务于定向更新与技能提炼，减少仅靠额外探索寻找收益的需求。

图中横轴是 rollout 小计；元智能体的 token 消耗另行分析，不能将这张图直接解释成总成本最低。

图 4 • 更强元智能体也可能更贵

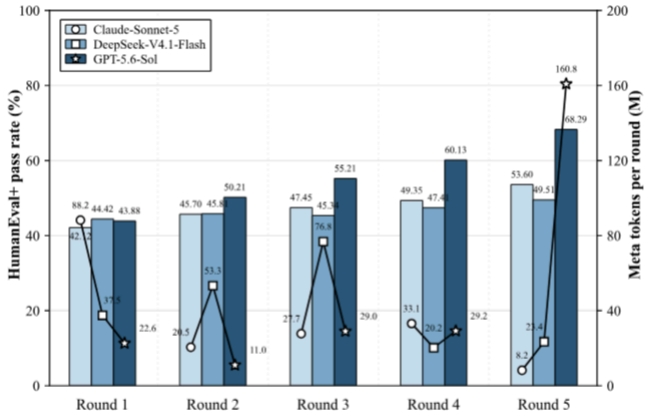


Figure 4: Sensitivity of RSIFlow to different Meta-Agent backbones on HumanEval+.

图 4：左轴为 HumanEval+，右轴为每轮元模型 token（M）。

HumanEval+ 上三种元模型都改善结果。GPT-5.6-Sol 最终达到 **68.29%**，但 API token 消耗更高；元模型能力会影响经验利用与累计收益。

图同时显示每轮性能与元模型 token，用于理解效果和成本的权衡，不能只看最后分数。

实验 • 局限 • 未来展望

表 3 • 元智能体自身是否进步?

Table 3: Domain-level performance of selected Meta candidates. Scores are macro-averages (%) across two benchmarks per domain. Component refers to the workflow selected for orchestration by meta agent.

Meta	Component	Code	Tool Use	SearchQA
Vanilla		48.25	29.70	34.52
Meta0	HARNESS	48.30	30.40	44.62
Meta1	HARNESS	46.32	31.16	45.04
Meta2	MODEL	67.90	31.51	43.99
Meta3	ARTIFACTS	67.08	**31.83**	50.58
Meta4	HARNESS	**68.86**	30.84	**54.13**

表 3：相同初始 Task-Agent；每个领域分数为两个基准的宏平均。

让不同 Meta 版本改进同一个初始 Task-Agent，可减少任务模型进步带来的混淆。Meta0、Meta4 都选择 Harness，但 Code 从 48.30 升到 68.86，SearchQA 从 44.62 升到 54.13。

Tool Use 最高的版本是选择 Artifacts 的 Meta3（31.83）；较晚元版本并非在所有领域、每一轮都最好。

图 5a • 五轮更新目标如何变化?

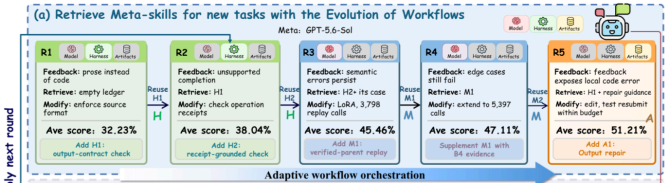


图 5a：H–H–M–M–A 的五轮实例，技能随更新结果积累。

示例顺序为 **Harness → Harness → Model → Model → Artifacts**：先修输出格式与完成凭据，再针对语义错误做 LoRA，最后根据执行反馈修代码。

五轮报告分数依次为 32.23、38.04、45.46、47.11、51.21。这里说明一条演化谱系，不把它当成所有任务都遵循的固定顺序。

图 5b • 技能如何被复用与修订?

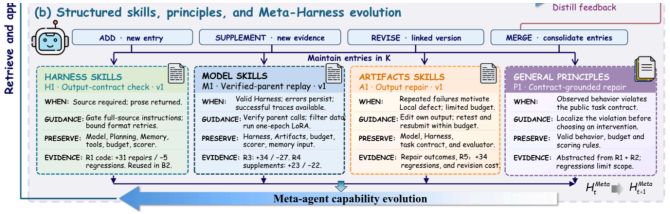


图 5b：组件技能与跨组件原则，通过 Add、Supplement、Revise、Merge 维护。

H1 的输出契约检查用于后续诊断；M1 的成功父代轨迹重放被复用并补充新证据；最终生成产物修复技能。记录中同时保存修复与退化，限制技能的适用范围。

局限

收益尚未扩展验证到更大任务模型或集群规模；元层演化带来额外开销，其长期稳定性仍需研究。

规模覆盖与成本

主实验仍是 Qwen3-4B、8B。更大模型或集群规模上的显著收益和可扩展性尚未验证。

更新 Meta-Harness 会增加计算开销，而换低成本元模型可能牺牲演化收益；经验驱动更新的稳定性也没有充分验证。

未来展望

作者计划扩大规模，并用更精确的元层诊断和局部修改提高演化效率、稳定性。

作者明确提出的方向

扩展到更大规模设置，检验 RSIFlow 的可扩展性。

通过反事实分析定位元智能体缺陷，开展针对性局部修改，并研究元模型参数优化。后者是未来方向，当前方法没有更新元模型权重。